

Documento Global del Proyecto PlataformaGame

Este documento resume el funcionamiento completo del proyecto: arquitectura, configuración de escenas, jugador, enemigos, UI (pausa/menú/inventario), parallax, tilemaps y condiciones de victoria/derrota. Sirve como guía de referencia rápida para reusar conceptos en otros proyectos.

1. Arquitectura general

- Núcleo: **GameManager** (Singleton)
 - Responsabilidades: pausa/reanudar, reiniciar nivel, cálculo de objetivos de monedas/enemigos, victoria/derrota, volver a menú, salir del juego.
 - Configuración: **coinsGoalMode**, **autoComputeCoins/Enemies**, **coinsRoot/enemiesRoot**, **includeInactiveInCounts**, **mainMenuSceneName**.
- UI: **HUD** (Singleton)
 - Muestra contadores, escucha eventos, ofrece métodos para botones: **UI_Resume**, **UI_RestartLevel**.
 - No contiene lógica de juego; solo presentación y callbacks.
- Jugador: **CharacterController**
 - Centraliza input de teclado (ESC, movimiento, salto, disparo, ataque).
 - Aplica física (saltos, impulso por golpes), anima estados y reporta muerte.
- Enemigos: **Enemigo**
 - Vida/daño, animaciones, muerte con **EnemyKilled** y destrucción.
 - Comportamientos extra: **EnemyPatrol2D**, persecución, etc.
- Recolectables: **Coin** con **valor**.
- Audio: **AudioManager** para SFX.
- PubSub: **EventBus<T>** + **GameEvent** para desacoplar UI/lógica.

2. Escenas y flujo

- Menú principal (MainMenu):
 - Canvas con botones: Jugar (carga **Level_01**), Salir (**GameManager.QuitGame()**).
 - Fondo/animaciones opcionales.
- Niveles (Level_01, ...):
 - Contienen GameManager, HUD, jugador, enemigos y tilemaps (Ground/OneWay/Back/Front).
 - Contenedores **coinsRoot** y **enemiesRoot** en la raíz para agrupar monedas y enemigos.
- Pausa:
 - ESC desde **CharacterController** → **GameManager.TogglePause()**.
 - Panel de pausa (Canvas) con botones conectados a HUD/GameManager.

3. GameManager (metas y flujo de juego)

- Objetivos:
 - **coinsGoalMode**: Manual | ByValue | ByCount.
 - **autoComputeCoins/autoComputeEnemies**: si ON, el objetivo se calcula en **Awake()** antes de mostrar la UI.

- `coinsRoot/enemiesRoot`: si se asignan, el cálculo busca solo debajo de estos contenedores (más rápido).
- `includeInactiveInCounts`: incluye objetos desactivados para escenarios donde se activan después.
- Progreso:
 - `CoinCollected` suma a `coinsCollected` (valor o 1 según el modo elegido).
 - `EnemyKilled` suma a `enemiesKilled`.
 - `CheckWin()` compara con `targetCoins/targetEnemies` y publica `WinConditionMet` cuando se cumple.
- Contrato (inputs/outputs):
 - Inputs: eventos `CoinCollected(int)`, `EnemyKilled(int)`, entradas de UI (botones HUD), tecla ESC vía `CharacterController`.
 - Outputs: eventos `GamePaused/Resumed`, `LevelReset`, `WinConditionMet`, `PlayerDied`.
- Errores comunes:
 - Más de un `GameManager` en escena → revisa Singleton en Awake.
 - `mainMenuSceneName` no en Build Settings → no carga.
 - `coinsRoot/enemiesRoot` sin hijos → objetivos en 0.
- Pausa y reinicio:
 - `TogglePause()`, `Pause()`, `Resume()`.
 - `ResetLevel()` recarga la escena actual.
- Salida:
 - `ExitToMainMenu()` carga `mainMenuSceneName`.
 - `QuitGame()` cierra la aplicación (o detiene Play en Editor).
- Muerte del jugador:
 - `TriggerPlayerDeath()` publica `PlayerDied` y congela `Time.timeScale`.

4. HUD (UI del juego)

- Textos: monedas, enemigos, (opcional) indicador de pausa.
- Eventos escuchados: `CoinCollected`, `EnemyKilled`, `GamePaused`, `GameResumed`, `LevelReset`, `WinConditionMet`.
- Botones del panel de pausa conectan a: `HUD.UI_Resume()`, `HUD.UI_RestartLevel()`, `GameManager.Instance.ExitToMainMenu()`, `GameManager.Instance.QuitGame()`.
- Recomendación: usar `Button` nativo para `OnClick` y `transitions`; añadir scripts de interfaces solo para efectos extra (hover/sonido/escala/tooltips).
- Checklist HUD:
 - Asignar textos `puntos`, `enemigosText`, `pausaText`.
 - Panel de pausa desactivado por defecto.
 - Botones conectados a `HUD/GameManager` como se detalla.
 - `EventSystem` y `GraphicRaycaster` presentes en la escena.

5. Jugador (CharacterController)

- Input centralizado (Update):
 - ESC → pausa.
 - Horizontal (teclas A y D) → movimiento.
 - Space → salto (con contador de saltos).

- R → disparo (`Weapon2D.TryFire()`).
- F → ataque (trigger de animación).
- Colisiones:
 - `deathMask`: si colisiona, llama `GameManager.TriggerPlayerDeath()`.
 - `enemyWeaponMask`: recibe daño al entrar en hitbox de enemigo.
- Configuración del jugador:
 - `velocidad`, `fuerzaSalto`, `saltosMax`, `fuerzaGolpe`.
 - Capas: `capaSuelo`, `deathMask`, `enemyWeaponMask`.
 - Referencias: `Weapon2D muzzle`, `AudioClip audioSalto`.
 - Animator: parámetros `isRunning`, `TriggerJump`, `TriggerAttacked`, `TriggerAttacking`.

6. Enemigos

- Script base: `Enemigo` con vida, animaciones y detección de impacto de balas.
- Al morir, publica `GameEvent.EnemyKilled` y se destruye.
- Configuración:
 - `vida`, sonidos de daño/muerte.
 - Asignar `playerWeaponMask` para validar qué capas hacen daño.
 - Añadir `EnemyPatrol2D/behaviours` de IA según sea necesario.
- Cómo añadir un enemigo:
 1. Crea un prefab con `SpriteRenderer`, `Rigidbody2D`, `Collider2D`, `Animator` y `Enemigo`.
 2. Ajusta `vida`, máscaras y clips de audio.
 3. Añade `EnemyPatrol2D` si patrulla; configura waypoints.
 4. Colócalo bajo `enemiesRoot` en la escena (para cómputo de objetivos).

7. Monedas y recolectables

- `Coin` (`OnTriggerEnter2D` con `Player` tag): publica `CoinCollected` con `valor` y se destruye.
- Configura el valor en el Inspector.
- Agrupa bajo `coinsRoot` para un cómputo rápido en `GameManager`.
- Cómo añadir monedas:
 1. Prefab con `SpriteRenderer`, `Collider2D` (`isTrigger`), y `Coin`.
 2. Configura `valor` y `AudioClip`.
 3. Colocar bajo `coinsRoot`.

8. UI: botones e interfaces

- Button (nativo): usa `Transition = Color Tint/Sprite Swap/Animation` para feedback.
- Interfaces (EventSystems): `IPointerEnter/Exit`, `IPointerDown/Up`, `IPointerClick`, `IBeginDrag/IDrag/IEndDrag`, `IDrop`, `IScroll`, `ISelect/IDeselect`, `IMove`, `ISubmit/ICancel`.
- Patrones: tooltips con hover, drag&drop de inventario, navegación por teclado/gamepad, press&hold.
- Requisitos para eventos UI:
 - `EventSystem` en escena; `GraphicRaycaster` en el Canvas.
 - Elementos con `Graphic.raycastTarget = true` para recibir eventos.
 - En World Space: asigna `Canvas.worldCamera` y sorting adecuado.
- Decisiones:
 - Color Tint (barato) vs Animation (flexible) vs Script (control fino sin Animator).

9. Inventario (básico)

- Modelo: **InventoryModel** (ScriptableObject) con slots y stacks.
- Item: **ItemData** con id, nombre, icono, maxStack.
- UI: **InventoryUI** crea slots (GridLayout) y reacciona a hover/click.
- Extensiones sugeridas: drag&drop, hotbar, tooltips, usar/consumir con click derecho.
- Contrato inventario:
 - Add(ItemData, cantidad) devuelve bool si cupo.
 - Remove(itemId, cantidad) idem.
 - Evento **OnChanged** para refrescar UI.
- UI de slots:
 - Prefab Slot (Image fondo + Image icono + Text cantidad).
 - **InventoryUI** instancia y conecta **InventoryUISlot** que maneja hover/click con EventSystem.

10. Parallax (resumen)

- Capas de fondo con distinto factor de movimiento relativo a la cámara.
- Canvas y sprites en sorting layers apropiadas (Back/Background/Foreground).
- En 2D: mover capas en X según delta de cámara; en vertical, también Y si el nivel lo requiere.
- Ajusta "clamp" o repetición si necesitas scroll infinito.
- Esquema rápido:
 - ParallaxLayer: script por capa con factor (0..1). 0 = fijo, 1 = sigue cámara.
 - En **LateUpdate**, mover **transform.position** en proporción al delta de la cámara.
 - Repetición: duplicar sprites a los lados y reposicionar cuando salgan del rango.

11. Tilemaps y grilla

- Define una celda base (16×16 o 32×32 px) y ajusta el PPU del spritesheet.
- Corta el tileset en el Sprite Editor por rejilla de la base.
- Para piezas grandes (ventanas/columnas), componer con múltiples celdas para mantener colisiones limpias.
- Tilemap_Ground: **TilemapCollider2D + CompositeCollider2D + Rigidbody2D Static** (Used by Composite ON).
- Tilemap_OneWay: plataformas de paso con **PlatformEffector2D**.
- Tilemap_Back/Front: decoración sin colisión.
- Organización recomendada:
 - Grid raíz (Rectangular, Cell Size acorde a PPU).
 - **Tilemap_Ground** (CompositeCollider2D + Rigidbody2D Static, Used by Composite).
 - **Tilemap_OneWay** (TilemapCollider2D + PlatformEffector2D).
 - **Tilemap_Back, Tilemap_Front** (solo render).
- RuleTiles:
 - Usa 2D Tilemap Extras para esquinas/bordes automáticos.
 - Mantén consistencia de trims (amarillo/verde) para lectura de "piso válido".

12. Condiciones de victoria y derrota

- Victoria: alcanzar **targetCoins** (según modo) y **targetEnemies** → **WinConditionMet** publicado, juego pausado.

- Derrota jugador: colisión con `deathMask` o vidas a 0 → `TriggerPlayerDeath()`.
- Reinicio: botón en pausa → `GameManager.ResetLevel()`.
- Errores típicos:
 - Objetivos en 0 por no asignar contenedores o por tener `coinsGoalMode=Manual` sin definir `targetCoins`.
 - `deathMask` mal configurada → no detecta zonas de muerte.

13. Configuración rápida por escena

1. Añade `GameManager` y configura:
 - `coinsGoalMode`, `autoComputeCoins/Enemies`, `includeInactive`.
 - `coinsRoot/enemiesRoot` (asigna los contenedores en Hierarchy).
 - `mainMenuSceneName`.
2. Añade `HUD` y asigna textos (monedas, enemigos, pausa).
3. Panel de Pausa (Canvas): botones → `HUD.UI_Resume`, `HUD.UI_RestartLevel`, `GameManager.Instance.ExitToMainMenu`, `GameManager.Instance.QuitGame`.
4. Jugador: `CharacterController` con referencias (`Weapon2D`, capas de suelo y muerte, audio de salto, etc.).
5. Enemigos bajo `enemiesRoot`, Monedas bajo `coinsRoot`.
6. Tilemaps: Ground con Composite, OneWay con Effector, Back/Front decorativos.
- 7. Parallax: añade capas con su factor, ordena sorting layers y prueba desplazamiento de cámara.
- 8. Audio: asigna `AudioManager` y clips clave (salto, moneda, daño, muerte).

14. Eventos (GameEvent)

- `CoinCollected` (int valor)
- `EnemyKilled` (int count)
- `VidaGanada` (int antesDeSumar)
- `VidaPerdida` (int vidasRestantes)
- `GamePaused` / `GameResumed` (bool)
- `LevelReset` (bool)
- `WinConditionMet` (bool)
- `PlayerDied` (bool)
- Nota: Si agregas eventos nuevos, actualiza HUD/gestores que los escuchen.

15. Consejos de mantenimiento

- Evita duplicados: un solo `GameManager` y un solo `HUD` por escena.
 - Usa contenedores (`coinsRoot/enemiesRoot`) para minimizar búsquedas.
 - Separa colisiones (Ground/OneWay) de decoración (Back/Front) para composites limpios.
 - Documenta convenciones de capas, sorting layers y tags.
 - Versionado: Commits pequeños y descriptivos ("GM: compute goals", "HUD: pause UI").
 - Prefabs: Prefabs para enemigos/monedas/paneles para reutilizar.
 - Performance: usa contenedores (roots) y `CompositeCollider` para reducir colliders.
-

Para detalles ampliados de UI (botones, interfaces) y del inventario, consulta también

[Docs/GUIA_UI_INVENTARIO.md](#). Este documento intenta ser la vista unificada con todos los puntos clave del proyecto.